PROPIEDADES BÁSICAS DE LA ORIENTACIÓN A OBJETOS

ABSTRACCIÓN Y ENCAPSULAMIENTO

Abstracción:

Representación de las características fundamentales de algo sin incluir antecedentes o detalles irrelevantes. Es uno de los métodos fundamentales para enfrentarse a la complejidad inherente al software.

El elemento clave de la abstracción en la Orientación a Objetos es la clase. Una clase describe un grupo de objetos identificando su estado y comportamiento.

Encapsulamiento:

Proceso de ocultación al exterior de aquellos detalles de la implementación de un objeto que no es necesario conocer para su uso.

En ocasiones también se usa para hacer referencia al proceso de almacenar en un mismo compartimento los elementos de una abstracción que constituyen su estructura y su comportamiento.

Abstracción vs. Encapsulamiento

- Son conceptos complementarios.
- La abstracción se centra en el comportamiento observable de un objeto.
- El encapsulamiento se centra en cómo implementar el objeto que da lugar a ese comportamiento.

Ventajas

- **Ocultación de la información**: La supresión de los detalles de bajo nivel nos permite razonar acerca de la operación u objeto de forma más eficiente. Un cambio en la representación interna, si no afecta a la parte publica de un objeto, no afectará a los clientes que usen dicho.
- **Representación conjunta de estado y comportamiento**: Todo lo relacionado con un objeto determinado está situado en la clase software que lo representa.

MODULARIDAD

Propiedad que tiene un sistema que ha sido descompuesto en un conjunto de partes o módulos que sean cohesivos (guardan cierta relación lógica) y débilmente acoplados (minimizan sus dependencias).

Ventajas

- El hecho de fragmentar un programa en componentes individuales suele contribuir a reducir su complejidad.
- Permite crear una serie de fronteras bien definidas y dentro del programa.
- Aumenta la comprensión del mismo.

Relación con el encapsulamiento

Los módulos pueden ocultar información a otros módulos actuando como mecanismos de encapsulamiento a un nivel superior al encapsulamiento de las propias clases.

@informaticapau

JERARQUÍA

Una jerarquía es una clasificación de las abstracciones.

Existen dos tipos de jerarquías en la programación orientada a objetos:

- Composición.
- Herencia.

Composición

- Define relaciones "TIENE UN" y ocurre cuando un elemento contiene otros elementos (los objetos incluyen otros objetos en la definición de su estado).
- Los objetos más simples tienen su estado compuesto de tipos básicos u otros objetos que juegan un rol similar.
- Los objetos compuestos incluyen también en su estado a otros objetos. Habitualmente los objetos compuestos reciben peticiones cuya resolución se delega a alguno de los objetos internos.

Herencia

- Relación "ES UN" entre clases, en las que una clase hereda la estructura y el comportamiento definidos en una o más clases.
- Una subclase hereda de una o más superclases y aumenta o redefine la estructura y el comportamiento de dichas superclases.
    - Las subclases representan conceptos especializados.
    - Las superclases representan generalizaciones de los aspectos comunes de las subclases.
    - La relación de herencia puede ser una relación de generalización o especialización.

Estructuras de herencia

La estructura final de las relaciones de herencia depende de aspectos como la herencia simple o múltiple y las clases raíz:

- **Herencia simple**: Cada clase tiene, como máximo, un ancestro.
    - Ventajas: Las estructuras de herencia son sencillas y tienen forma de árbol.
- **Clase raíz**: Clase que actúa como superclase del resto de clases. En combinación con la herencia simple genera una estructura de árbol con una raíz única.
- **Herencia múltiple**: Una clase puede heredar de varias clases simultáneamente.
    - Ventajas:
        - Una clase puede heredar características de varias superclases no relacionadas.
        - Más posibilidades de diseño.
    - Inconvenientes:
        - Genera conflictos cuya resolución es específica de cada lenguaje.
        - Aumenta la complejidad de la programación y reduce la comprensibilidad.

La tendencia actual es evitar la herencia múltiple ya que es posible desarrollar jerarquías de herencia complejas y flexibles con herencia simple.

@informaticapau

Herencia y constructores

Un objeto de una subclase tiene siempre un objeto de una superclase dentro. Antes de crear el objeto de la subclase se crea primero el objeto de la superclase.

Herencia vs composición

Tanto la herencia como la composición pueden usarse como mecanismos de reutilización, construyendo nuevas clases basándose en clases ya creadas.

Ventajas de la herencia

- **Lleva implícito el principio de sustitución**: Las subclases son instancias de las superclases y pueden pasarse ahí donde se pida una superclase.
- **El código a escribir es menor**: Al heredar de una clase automáticamente heredamos sus métodos y atributos. No se necesitan mecanismos de delegación para acceder a ellos (con lo que se evitan rellamadas).
- **Permite el acceso a elementos protegidos**: La visibilidad protegida está pensada para dar acceso a esos elementos a las subclases, pero no a otras clases.
- **Las relaciones entre clases son más públicas**: Que una clase herede de otra forma parte de su interfaz público.

Ventajas de la composición

- **Indica con claridad cuáles son las operaciones que podemos utilizar en la nueva estructura de datos definida**:
    - La herencia es menos clara de comprender porque obliga a analizar la estructura de herencia para comprender mejor el funcionamiento de la clase creada.
    - La composición permite no heredar métodos indeseados.
- **Es más fácil de modificar**: Si el objeto interno es privado es fácil cambiarlo por otro afectando sólo a la clase local y no a otras clases que lo usen.
- **No implica sustitución**: La composición es una relación “TIENE UN” y no “ES UN”, por lo que evita el principio de sustitución propio de la herencia.
- **Es una solución más general**: La herencia tiene limitaciones (herencia simple vs. múltiple) que no tiene la composición (podemos crear una composición de varias clases y usar sólo aquellos métodos que nos convengan).

Conclusiones

- Usar herencia exclusivamente en aquellas relaciones que claramente corresponden a un tipo “ES UN” y en las que tiene sentido pasar un elemento de la subclase como un elemento de la superclase.
- Usar composición en aquellas relaciones que no se correspondan a un tipo “ES UN” claro.
- En caso de duda favorecer siempre la composición sobre la herencia.

Clases abstractas

Clase que no pueden ser instanciada (no se puede crear un objeto de la misma) y que está destinada a ser extendida por herencia.

Objetivos de las clases abstractas

- Definir clases generales que agrupan características comunes: Son clases destinadas a la herencia y de las cuales no tiene sentido que existan instancias.
- Diferir comportamiento a las subclases: Hay métodos cuyo interfaz podemos definir en la superclase, pero cuya implementación es propia de cada una de las subclases.

@informaticapau

- Asegurar la consistencia en el interfaz de todas las subclases: Las subclases están obligadas a implementar los métodos heredados de una clase abstracta, eso permite asegurar que todas las subclases tienen una implementación de dicho método con el interfaz definido en la superclase.

Métodos abstractos

Método en el que sólo se define su interfaz (nombre, parámetros y tipo de retorno) pero no su implementación (que se difiere a las subclases).

Un método abstracto sólo puede pertenecer a una clase abstracta, pero una clase abstracta puede tener métodos no abstractos.

Las subclases de una clase abstracta deben implementar los métodos abstractos o declararse como abstractas.

Interfaces

Son clases abstractas "puras" que definen un protocolo de conducta, pero no cómo debe implementarse dicha conducta.

Contienen las cabeceras de métodos que son, implícitamente, públicos y abstractos, por lo que no incluyen los cuerpos de dichos métodos.

Pueden contener atributos, pero serán implícitamente públicos, estáticos y finales, es decir, constantes de clase.

Un interfaz no se diferencia mucho de una clase abstracta con todos sus métodos abstractos.

Se pierde la capacidad de incluir atributos y métodos no abstractos.

Herencia entre interfaces

Un interfaz puede heredar de otros interfaces (incluye herencia múltiple), pero nunca de una clase.

- En este caso la herencia múltiple no es un problema ya que los interfaces sólo tienen la cabecera de los métodos, no la implementación, por lo que no se corre el riesgo de heredar implementaciones diferentes para un mismo método.
- La herencia de interfaces es paralela a la herencia entre clases. La conexión entre ambas jerarquías de herencia se hace mediante el proceso de implementación de interfaces.

Implementación de interfaces

- Implementar un interfaz consiste en darle una implementación concreta a todos y cada uno de los métodos que contiene.
- Las clases pueden implementar múltiples interfaces.
- Una clase puede evitar dar una implementación a los métodos del interfaz declarándose abstracta.
- También se puede realizar una implementación temporal y dejar la implementación definitiva para más adelante.

Usos de los interfaces

Usos típicos de los interfaces:

- Revelar el interfaz de programación de un objeto sin revelar su clase.
    - Un interfaz revela un servicio que ofrece una clase, pero no indica qué clase en concreto está ofreciendo dicho servicio.

@informaticapau

    - Gracias a esa ignorancia de qué clase está realizando la implementación, dicha clase puede ser cambiada dinámicamente sin afectar al cliente que utiliza el servicio.
- Capturar las similitudes entre clases no relacionadas.
    - Clases que no están relacionadas en absoluto a través de la herencia pueden ser consideradas como instancias de la misma clase si implementan el mismo interfaz:
    - Si una clase implementa un interfaz se considera que un objeto de esa clase es una instancia de dicho interfaz.
- Definir nuevos tipos de datos.
    - Un interfaz representa un nuevo tipo de datos. En cualquier momento se pueden añadir nuevas instancias diferentes de dicho tipo de datos simplemente definiendo nuevas clases implementadoras del interfaz.

Usos poco recomendados de los interfaces

- Marcadores de clase.
    - Un interfaz que se define sin incluir ningún método ni atributo es conocido como un interfaz marcador de clase.
    - Como no incluye ningún elemento las clases que implementan este ´ interfaz no están obligadas a hacer nada (ni heredan nada) pero quedan marcadas por dicho interfaz.
- Contenedores de elementos globales.
    - Los programadores usan a veces a los interfaces como repositorios de constantes.
    - Problemas:
        - Los interfaces que implementa una clase es algo público y notorio, se muestra en dicho interfaz un aspecto de la implementación.
        - Se crean dependencias.
    - Soluciones:
        - Crear las constantes como tipos enumerados.
        - Crear las constantes en clases relacionadas.
        - Crear clases de utilidad para alojar dichas constantes.
            - Nuevo problema: en cada uso de la constante hay que anteponer el nombre de la clase.
            - Utilizar la importación estática.

POLIMORFISMO

- Ad Hoc o Aparente: El polimorfismo es sólo aparente. Las funciones realmente sólo tratan parámetros de un único tipo, pero simulan aceptar varios tipos.
    - Sobrecarga.
    - Coacción.
- Universal o Verdadero: Existen objetos (variables) que pertenecen a distintas clases (tipos). Existen funciones polimórficas que usan el mismo código para tratar a los objetos de distintas clases.
    - De Inclusión.
    - Paramétrico.

@informaticapau

Sobrecarga (*overloading*)

Consiste en utilizar el mismo nombre para denotar métodos distintos diferenciándolos por el numero o tipo de los parámetros.

- Sobrecarga paramétrica: Sobrecarga que involucra a métodos de una misma clase que se diferencian por el número y tipo de parámetros.
- Sobrecarga entre clases: Sobrecarga que involucra a métodos situados en distintas clases y que suelen tener los mismos parámetros (el parámetro que cambia es el objeto sobre el que son llamados). La sobrecarga entre clases se usa para dar nombres comunes a operaciones comunes y facilitar el aprendizaje de bibliotecas de funciones.

Sobrescritura (*overriding*)

Situación que ocurre cuando una clase hija le da una implementación más específica a un método implementado y heredado de una clase padre.

Para que la sobrescritura sea efectiva la firma de los dos métodos debe ser igual (nombre, parámetros, etc.).

Variaciones permitidas

- El método de la subclase puede devolver un objeto que es subclase del tipo de retorno de la superclase.
- El método de la subclase puede cambiar la visibilidad del método, pero sólo a una más permisiva.

Sobrescritura y sobrecarga

La sobrescritura en un primer momento puede parecer una sobrecarga entre clases (mismo método con mismo número de parámetros en dos clases distintas), pero al haber una relación de herencia entre las dos clases involucradas el método de la subclase sobrescribe al método de la superclase.

La implementación más específica es preferida sobre la implementación de la clase padre al estar más adaptada al objeto actual.

Coacción (o Coerción)

Operación semántica por la cual se convierte un argumento al tipo esperado por una función para evitar que se produzca un error de tipos.

Las coacciones pueden hacerse estáticamente, insertándolas entre los argumentos y las funciones en tiempo de compilación (serían los conocidos *type cast* o conversiones de tipos), o pueden ser determinadas dinámicamente por tests en tiempo de ejecución sobre los argumentos.

Polimorfismo de inclusión (o polimorfismo de subtipos)

Polimorfismo que ocurre a través de las relaciones de herencia y mediante el cual una instancia de una subclase es también una instancia de sus superclases. Es el polimorfismo típico de la POO.

El polimorfismo de inclusión permite que un objeto de una subclase puede utilizarse en aquellos lugares en los que se requiere un objeto de alguna de sus superclases. Lo contrario no es cierto, no podemos asignar un objeto de una superclase a una variable de las subclases.

@informaticapau

Métodos genéricos

En muchos lenguajes OO existe una superclase común para todas las clases. Es sencillo crear métodos genéricos que acepten parámetros de cualquier tipo o que devuelvan valores de cualquier tipo usando parámetros o tipos de retorno de dicha clase.

Clases envoltorio (Wrapper classes)

Clases que "envuelven" a un tipo básico y le proveen del interfaz de objeto que le hace falta para poder ir en una colección de objetos.

Son clases inmutables, que tienen un atributo privado del tipo básico que están conteniendo y métodos de lectura del mismo. Adicionalmente también incluyen métodos de utilidad.

- **Boxing**: Proceso de introducir un tipo básico en su correspondiente clase envoltorio.
- **Unboxing**: Proceso de extraer un tipo básico de su correspondiente clase envoltorio.

- Problema de las clases envoltorio: su utilización se hace muy engorrosa.
- Solución: autoboxing/autounboxing. Es una versión avanzada de la Coacción aplicada a tipos básicos y clases envoltorio.
    - **Autoboxing**: Proceso mediante el cual un tipo básico es convertido automáticamente en su clase envoltorio cuando se requiere que se comporte como un objeto.
    - **Autounboxing**: Proceso mediante el cual un objeto de una clase envoltorio es convertido automáticamente en su tipo básico cuando se requiere que se comporte como dicho tipo básico.

Polimorfismo paramétrico (genericidad)

Consiste en que una misma función es aplicada sobre una variedad de tipos distintos. La especificación de qué tipo concreto usar en cada momento se realiza a través de un parámetro extra que se pasa a dicha función.

Clase genérica

Clase que incluye un parámetro de tipo (T) que representa a un objeto cualquiera cuyo tipo real será especificado a la hora de crear instancias de dicha clase.

Parámetros de tipo

Son parámetros genéricos que especifican un tipo indeterminado.

Ventajas

- **Seguridad**:
    - Se realizan comprobaciones en tiempo de compilación sobre el tipo de la colección.
    - Se evitan errores en tiempo de ejecución.
- **Claridad**: Se evitan molestas conversiones de tipos al extraer un elemento de la colección.
- **Comprensibilidad**: Al definir la colección se especifica su tipo, por lo que resulta más comprensible el motivo por el cual se está creando la colección.

Tipos vinculados

Parámetros de tipo vinculados a una clase que hace de límite superior. El tipo genérico solo podrá ser instancia de cualquier subclase de la clase límite, o una instancia de la propia clase límite.

@informaticapau

TIPIFICACIÓN

Tipos

Un tipo es una caracterización precisa de las propiedades estructurales y de comportamiento que comparten una serie de entidades.

En los lenguajes OO un tipo se asocia con una clase, pero no siempre es así:

- Hay tipos que no son clases.
- En ocasiones distintas clases implementan el mismo tipo de datos.
- Puede ocurrir que una subclase no sea un subtipo.

Tipado estático

La comprobación de tipos se hace en tiempo de compilación.

- Ventajas:
    - Mejor rendimiento.
    - Más legibilidad del código
    - Los errores se detectan antes y más fácilmente.
- Inconvenientes:
    - El código es más rígido y verboso.

Tipado dinámico

La comprobación de tipos se hace en tiempo de ejecución.

- Ventajas:
    - Código más sucinto.
    - Más fácil de escribir y aprender.
- Inconvenientes:
    - Los errores tardan más en detectarse y es más complejo solucionarlos.

Tipado fuerte

Las reglas de tipos son estrictas.

- Ventajas:
    - Se detectan más fácilmente los errores.
- Inconvenientes:
    - Ciertas operaciones no podrán realizarse al considerarse erróneas.

Tipado débil

Las reglas de tipos son más flexibles.

- Ventajas:
    - Hay menos restricciones a la hora de escribir el código.
- Inconvenientes:
    - Código más frágil y de resultados más impredecibles.

LIGADURA DINÁMICA

Ligadura: Proceso que se encarga de ligar o relacionar la llamada a un método (o mensaje) con el código que se ejecuta finalmente.

Ligadura estática (o temprana)

Consiste en realizar el proceso de ligadura en tiempo de compilación según el tipo declarado del objeto al que se manda el mensaje.

@informaticapau

Ligadura dinámica (o tardía)

Consiste en realizar el proceso de ligadura en tiempo de ejecución siendo la forma dinámica del objeto la que determina la versión del método a ejecutar.

Métodos estáticos

Al tener ligadura estática los métodos estáticos no aprovechan todas las posibilidades que ofrece la OO.

De todas formas, hay situaciones en las que puede ser interesante plantearse hacer un método estático:

- Cuando el método no modifica el estado del objeto ni accede a sus atributos.
- Cuando se quiere llamar al método sin tener que disponer de una instancia de la clase.
- Cuando el funcionamiento del método sólo depende de los parámetros que le pasemos.
- Métodos de utilidad que podrían situarse en cualquier clase.
- Desarrollo de algunos patrones de diseño.

ESENCIA DEL DISEÑO OO

Usar las propiedades típicas de la OO (herencia, polimorfismo y ligadura dinámica) conjuntamente para obtener las ventajas que ofrece la OO: flexibilidad, escalabilidad, extensibilidad, abstracción, etc.

Propiedades:

- Herencia
- Polimorfismo
- Ligadura dinámica

Ventajas:

- Se crea código que será capaz de trabajar, sin modificarlo, con código futuro que aún no está escrito.
- Flexibilidad
- Escalabilidad
- Extensibilidad
- Abstracción
- Seguridad
- Facilidad de mantenimiento

@informaticapau